Activity No: 4.3	
Activity Title: Sorting and Searching Arrays	
Course Code: CPE-007	**Program: Computer Engineering**
Course Title: Programming Logic and Design	**Date Performed: September 1, 2026**
Section: CPE11S2	**Date Submitted: September 1, 2026**
Name(s): Jose, Redge Jansen A.	**Instructor: Sir Noel Jayson Lusung**

Objectives

2.1 Analyse and create programs that sorts arrays

2.2 Analyse and create programs that search arrays

Instructions

Output/Answer

1. **Write a program that asks for a number from the user and prints which day of the week that number corresponds to. The days are indexed from 0 (Sunday) to 6 (Saturday). Before the program gets a value from the array, it must first check if the given day is greater than or equal to zero and less than 7. If not, it should print the message: "Error, no such day." Your version of the program must print the same result as the expected output.**

Sample Input and Output:

Example input 0

Example output: Sunday

Example input 5

Example output: Friday

Example input 12

Example output: Error, no such day.

Answer/ScreenShot:

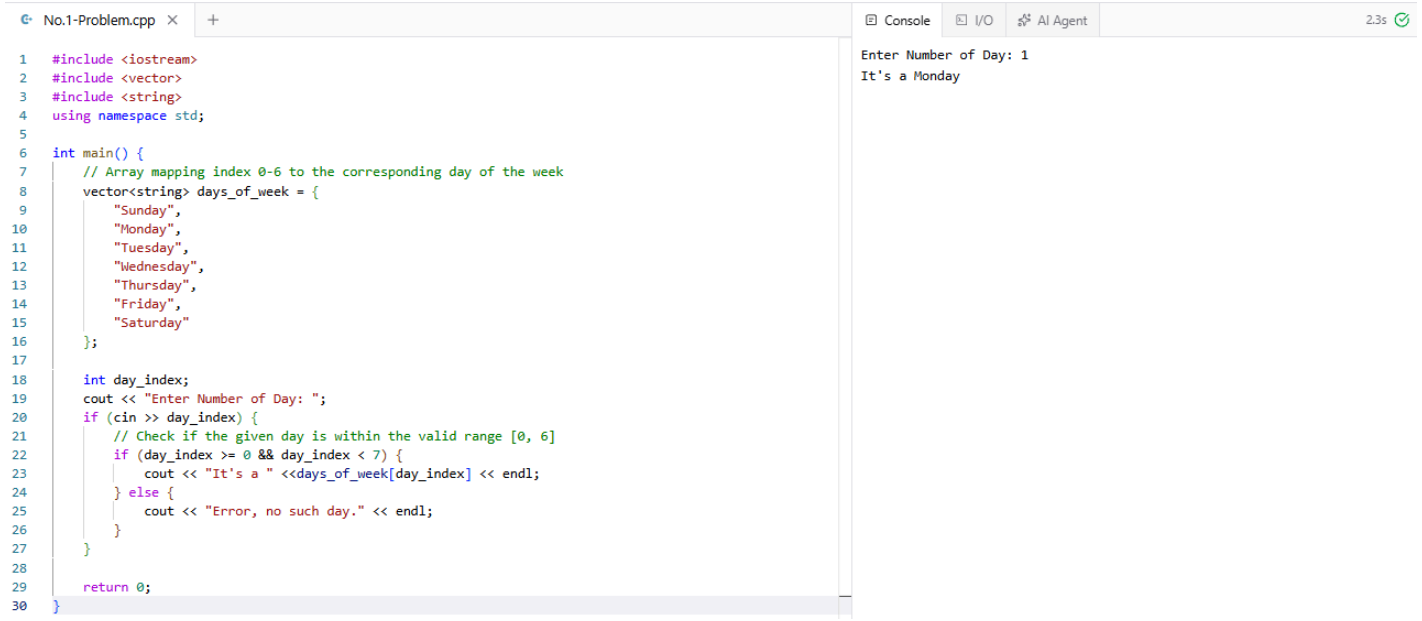
```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
   // Array mapping index 0-6 to the corresponding day of the week
   vector<string> days_of_week = {
      "Sunday",
      "Monday",
      "Tuesday",
      "Wednesday",
```

```
        "Thursday",
        "Friday",
        "Saturday"
    };

    int day_index;
    cout << "Enter Number of Day: ";
    if (cin >> day_index) {
        // Check if the given day is within the valid range [0, 6]
        if (day_index >= 0 && day_index < 7) {
            cout << "It's a " <<days_of_week[day_index] << endl;
        } else {
            cout << "Error, no such day." << endl;
        }
    }

    return 0;
}
```



2. **Write a program that creates a chessboard, sets all the pieces on it and then displays the contents of the board. Create a two-dimensional array, fill it with data and print a letter when a piece is on the field and a space when there is no piece. Store one letter for one piece. For now, we don't need any information about the color of the pieces. The starting positions (with letters which symbolize each piece) for all pieces are: The rooks (R) are placed on the outside corners, right and left edge (white on the 1st and black on the 8th line). The knights (N) are placed immediately inside of the rooks. The bishops (B) are placed immediately inside of the knights. The queen (Q) is placed on the central square of the same color as that of the player: white queen on the white square and black queen on the black square. Both stand on the d rank: white queen on the d1 field and black queen on the d8 field. The king (K) takes the vacant spot next to the queen. The pawns (P - not the official symbol, but you need to print something) are**

placed one square in front of all of the other pieces. Your version of the program must print the same result as the expected output.

Answer/ScreenShot:

```
#include <iostream>
#include <algorithm> // Required for std::sort
using namespace std;

// Function to print the 2D chessboard array
void displayBoard(char board[8][8]) {
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            cout << board[i][j];
            if (j < 7) {
                cout << " "; // Print space separation between columns
            }
        }
        cout << endl;
    }
}

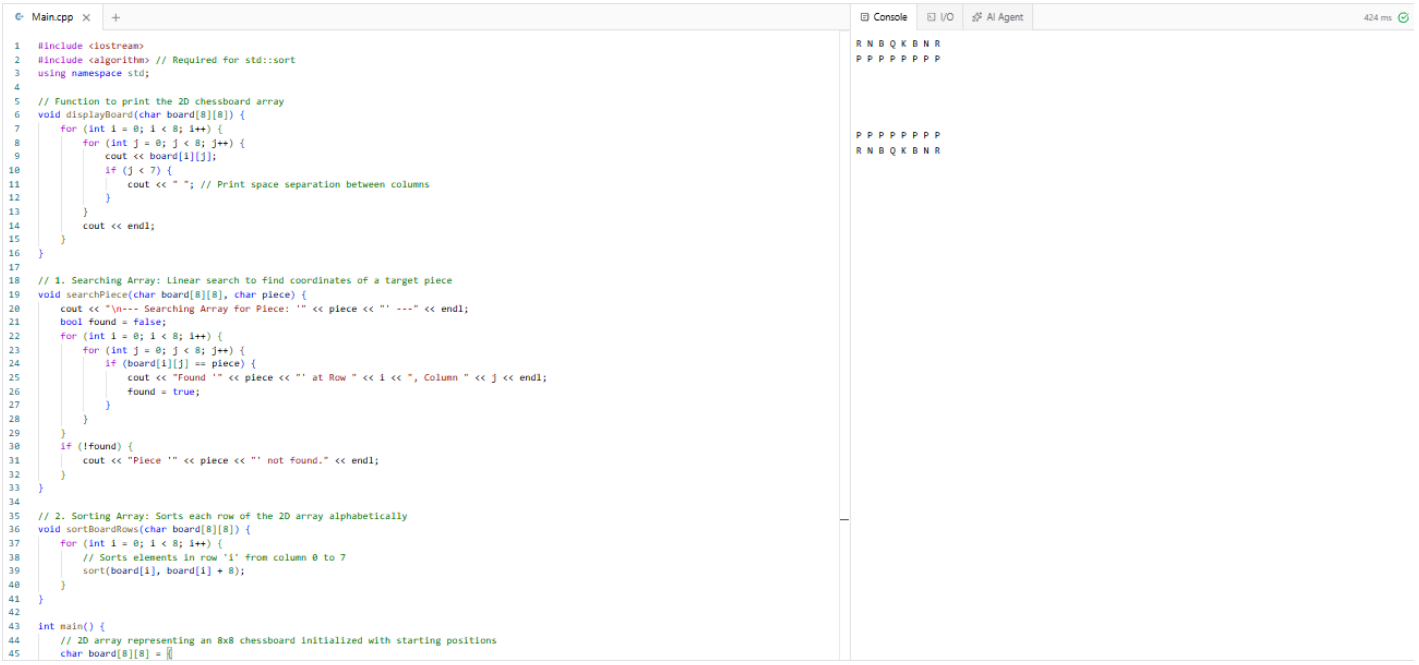
// 1. Searching Array: Linear search to find coordinates of a target piece
void searchPiece(char board[8][8], char piece) {
    cout << "\n--- Searching Array for Piece: '" << piece << "' ---" << endl;
    bool found = false;
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            if (board[i][j] == piece) {
                cout << "Found '" << piece << "' at Row " << i << ", Column " << j << endl;
                found = true;
            }
        }
    }
    if (!found) {
        cout << "Piece '" << piece << "' not found." << endl;
    }
}

// 2. Sorting Array: Sorts each row of the 2D array alphabetically
void sortBoardRows(char board[8][8]) {
    for (int i = 0; i < 8; i++) {
        // Sorts elements in row 'i' from column 0 to 7
```

```
        sort(board[i], board[i] + 8);
    }
}

int main() {
    // 2D array representing an 8x8 chessboard initialized with starting positions
    char board[8][8] = {
        {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'},
        {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'},
        {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'}
    };

    // Prints the exact target output required
    displayBoard(board);
    return 0;
}
```



Supplementary Activity (if any)

None
Conclusion
-Learning more about C++ in Sorting and Searching Arrays
Assessment Rubric: SO 7